SQL Window Function Practice Set (Beginner to Advanced)

Sample Dataset

1. Employees Table

```
CREATE TABLE employees (
    emp_id INT,
    emp_name VARCHAR(50),
    department_id INT,
    salary INT
);

INSERT INTO employees VALUES
(1,'Amit',10,50000),
(2,'Ravi',10,60000),
(3,'Neha',20,55000),
(4,'Priya',20,70000),
(5,'Arjun',30,65000),
(6,'Meera',30,62000);
```

2. Orders Table

```
CREATE TABLE orders (
    order_id INT,
    customer_id INT,
    order_date DATE,
    amount INT
);

INSERT INTO orders VALUES
(101,1,'2026-06-01',2000),
(102,2,'2026-06-02',1500),
(103,1,'2026-06-03',3000),
(104,3,'2026-06-04',2500),
(105,2,'2026-06-05',1800),
(106,3,'2026-06-06',2200);
```

3. Students Table

```
CREATE TABLE students (
    student_id INT,
    student_name VARCHAR(50),
    class_id INT,
    exam_date DATE,
    marks INT
);

INSERT INTO students VALUES
```

(1,'Rahul',101,'2026-06-01',78),
(2,'Sneha',101,'2026-06-02',85),
(3,'Karan',102,'2026-06-01',90),
(4,'Pooja',102,'2026-06-03',88),
(5,'Vikas',103,'2026-06-02',76),
(6,'Anita',103,'2026-06-04',82);

4. Products Table

```
CREATE TABLE products (
    product_id INT,
    product_name VARCHAR(50),
    category_id INT,
    sales INT
);

INSERT INTO products VALUES
(1,'Laptop',1,50000),
(2,'Tablet',1,30000),
(3,'Phone',2,40000),
(4,'Headphones',2,15000),
(5,'Camera',3,35000);
```

WINDOW FUNCTION QUESTIONS

Beginner (1-7)

1. Find each employee’s salary and average salary of their department using:
AVG() OVER(PARTITION BY department_id)

2. Show each order’s amount and running total using:
SUM() OVER(ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURR ENT ROW)

3. Display each student’s marks with highest marks in class using:
MAX() OVER(PARTITION BY class_id)

4. Calculate total sales for each product using:
SUM() OVER(PARTITION BY product_id)

5. Rank employees by salary within department using:
RANK() OVER(PARTITION BY department_id ORDER BY salary DESC)

6. Show order amount and cumulative average using:
AVG() OVER(ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURR ENT ROW)

7. Display student marks and row number within class using:
ROW_NUMBER() OVER(PARTITION BY class_id ORDER BY marks DESC)

Intermediate (8-14)

8. Find salary difference from department average.

9. Moving average of last 3 orders using:

AVG() OVER(ORDER BY order_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)

10. Product sales percentage contribution using:

SUM(sales) OVER(PARTITION BY category_id)

11. Dense rank employees across company:

DENSE_RANK() OVER(ORDER BY salary DESC)

12. Difference from previous order:

LAG(amount) OVER(ORDER BY order_date)

13. Next order amount:

LEAD(amount) OVER(ORDER BY order_date)

14. Running maximum marks:

MAX() OVER(ORDER BY exam_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURR ENT ROW)

Advanced (15-20)

15. Salary percentile rank:

PERCENT_RANK() OVER(ORDER BY salary)

16. Product cumulative distribution:

CUME_DIST() OVER(ORDER BY sales DESC)

17. Moving sum over last 7 days:

SUM(amount) OVER(ORDER BY order_date RANGE BETWEEN INTERVAL '7 days' PRECEDIN G AND CURRENT ROW)

18. Difference from class maximum:

MAX(marks) OVER(PARTITION BY class_id) - marks

19. Third highest salary in department:

NTH_VALUE(salary, 3) OVER(PARTITION BY department_id ORDER BY salary DESC)

20. Ratio to total company sales:

SUM(amount) OVER()